

SQL Portfolio: One-Page App Summary

Repository: /Users/andrew.findlay/Documents/Github/personal/website

What it is

SQL Portfolio is a single-page React/Vite app that presents a developer portfolio as a SQL IDE plus schema explorer (metadata.json). It runs DuckDB-WASM in the browser, seeds portfolio tables, and executes user SQL locally (lib/db.ts, constants.ts).

Who it is for

Primary persona: Not found in repo. Evidence suggests a developer/candidate showcasing experience to hiring teams (constants.ts sample resume data; ExportModal text: "Best for sharing with hiring teams").

What it does

- Initializes an in-browser DuckDB database and loads seeded schema/data for employees, history, education, and tech stack (lib/db.ts, constants.ts).
- Provides a multi-tab SQL editor with syntax highlighting, line numbers, and open/close tab behavior (components/Editor/CodeEditor.tsx, App.tsx).
- Runs SQL queries from the active file and tracks execution state and DB readiness in app state (App.tsx).
- Renders query results with column types, row numbers, execution time, affected-row count, and SQL error display (components/Editor/ResultsPane.tsx).
- Supports data export of current query results to CSV and JSON from the results toolbar (components/Editor/ResultsPane.tsx).
- Includes a schema explorer view driven by static schema nodes and relationship connectors (components/Schema/SchemaView.tsx, constants.ts).
- Includes an export modal UI for PDF/CSV/JSON selection and preview; modal download action wiring to actual file generation is Not found in repo (components/Modals/ExportModal.tsx).

How it works (compact architecture)

- Frontend shell: index.tsx mounts App; App composes Header, Sidebar/MobileNav, CodeEditor, ResultsPane, SchemaView, and ExportModal.
- State + orchestration: App.tsx controls active/open files, editor/schema view mode, modal visibility, run status, DB readiness, and per-file query results.
- Data service: lib/db.ts initializes @duckdb/duckdb-wasm in a Web Worker (bundle selection via jsDelivr), creates connection, and runs INIT_SQL.
- Data flow: user action -> App handlers -> runQuery(sql) -> Arrow result mapped to plain objects/columns -> ResultsPane render/export helpers.
- Config/runtime: Vite dev server (port 3000) and GEMINI_API_KEY define entries in vite.config.ts; backend API/service layer is Not found in repo.

How to run (minimal)

- Prerequisite: Node.js (README.md).
- Install dependencies: npm install.
- Set GEMINI_API_KEY in /Users/andrew.findlay/Documents/Github/personal/website/.env.local (README.md).
- Start dev server: npm run dev, then open http://localhost:3000 (vite.config.ts).